CONFIDENTIAL | INTERNAL USE ONLY

TalentConnect AI Matching Pipeline

Product Requirements Document

Version	1.0 - Working Draft
Date	17 March 2026
Module	Matching Algorithm
Status	Pre-implementation
Stack	Python, FastAPI, Gemini API, Chroma, XGBoost

Table of Contents

1. Problem Statement 4
2. Objectives and Success Metrics 4
2.1 Objectives 4
2.2 Success Metrics 4
3. Scope 4
3.1 In Scope 4
3.2 Out of Scope 5
3.3 System Context 5
4. User Stories 6
5. Functional Requirements 7
5.1 Embedding Generation 7
5.2 Semantic Similarity Search 7
5.3 Feature Extraction 8
5.4 Candidate Ranking 8
5.5 Behavioral Signal Collection 9
5.6 Feedback Processor and Iteration Logic 10
5.7 API Endpoints 11
6. Non-Functional Requirements 13
6.1 Performance 13
6.2 Reliability 13
6.3 Security 14
6.4 Maintainability 14
7. ML Training Module 15
7.1 Cold Start Dataset 15
7.2 Default Feature Weights 15
7.3 Training 15
7.4 Validation Criteria 15
7.5 Deployment 15
8. Data Contracts 16
8.1 Candidate Profile Schema 16
8.2 Job Post Schema 16
8.3 Match Result Schema 17
9. Monitoring and Alerting 18
10. System Diagrams 19
10.1 System Context 19
10.2 Matching Pipeline 20
10.3 Behavioral Feedback Loop 21
10.4 Adaptive Threshold Logic 22
11. Assumptions and Constraints 23

11.1 Assumptions 23
11.2 Constraints 23
12. Dependencies 24
12.1 Internal Dependencies 24
12.2 External Dependencies 24
12.3 Technology Stack 24
13. Open Items 25

1. Problem Statement

Hiring is broken in a specific, fixable way. Candidates tailor every application to every job post they can see. The result is a pile of optimised resumes that tells employers almost nothing about genuine skill. TalentConnect solves this by removing the job post from the candidate's view entirely. Candidates build one honest profile. Employers post requirements privately. A matching pipeline sits between them.

The pipeline must therefore recognize that a candidate who lists React and one who lists Vue.js are closer to each other than either is to a candidate who lists SAP. It must learn, over time, which types of profiles employers actually shortlist rather than which profiles employers say they want. And it must do all of this without exposing either party's data to the other prematurely.

2. Objectives and Success Metrics

2.1 Objectives

1. Produce semantically aware candidate rankings that surface genuinely qualified candidates.
2. Improve ranking quality over time through employer behavioral signals without requiring explicit employer ratings.
3. Return ranked results within latency bounds that keep the employer experience responsive.
4. Maintain a feedback loop that is self-correcting and does not require manual model tuning after initial deployment.

2.2 Success Metrics

Metric	Target	Measurement Method
NDCG@10 on held-out behavioral data	0.75 or above within 3 iteration cycles	Automated post-retrain evaluation
Employer shortlist rate	40% or more of presented candidates opened	Behavioral tracker Layer 1
Single match p95 latency	Under 200ms	Production APM
Bulk match completion (10K candidates)	Under 30 seconds	Load test suite
Model staleness	No model older than 30 days in production	Staleness cap monitor

3. Scope

3.1 In Scope

5. Vector embedding generation for candidate profiles and job posts.
6. Semantic similarity search producing an initial candidate shortlist.
7. Structured feature extraction and XGBoost-based candidate ranking.
8. Behavioral signal collection from employer interactions.
9. Adaptive threshold logic governing model iteration cycles.
10. XGBoost model retraining and validation pipeline.
11. Internal API endpoints exposing matching functionality to the backend team.

3.2 Out of Scope

12. User authentication and session management.
13. File upload and cloud storage infrastructure.
14. Document text extraction from PDFs, DOC, and DOCX files.
15. Video transcription from candidate introduction recordings.
16. Frontend rendering of match results.
17. Employer and candidate profile creation.
18. Relational database schema creation. We specify what we need; the backend team creates the tables.
19. Notification and invitation workflows.
20. Any employer-facing or candidate-facing UI.

3.3 System Context

The diagram below shows the boundary between our module and the surrounding system.

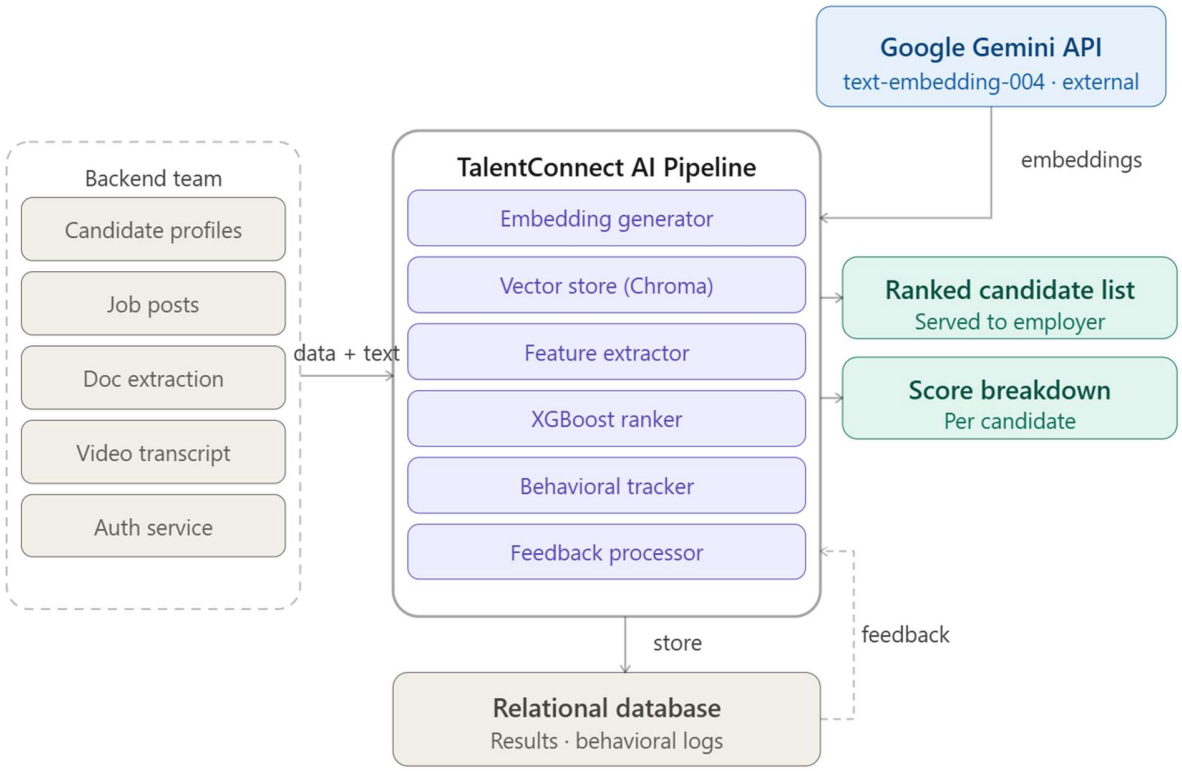

Figure 1. The TalentConnect AI Pipeline within the broader system. The dashed region denotes backend team responsibility. The solid border denotes Talent Connect Matching Algorithm. The Gemini API is an external dependency we call but do not host.

4. User Stories

ID	Story	Acceptance Criteria
US-01	As a candidate, I want my profile to be evaluated on the meaning of my skills rather than exact keyword matches, so that related experience is recognised.	A candidate with Vue.js experience scores above zero for a React role. A candidate with no frontend skills scores zero.
US-02	As an employer, I want the candidates presented to me to be ranked by genuine fit to my requirements, so that I spend less time filtering.	The top 5 candidates returned for any job post each satisfy at least 60% of the required skills.
US-03	As an employer, I want the system to learn from which candidates I actually shortlist, so that future rankings reflect my real preferences.	After 3 iteration cycles, NDCG@10 on behavioral data for the same employer category improves by at least 5% over baseline.
US-04	As a candidate, I want my profile to be re-evaluated immediately after I update it, so that employers always see my current skills.	A profile update triggers embedding regeneration within 60 seconds.
US-05	As the backend team, I want a well-defined API I can call to trigger matching, so that I do not need to understand the internal pipeline.	POST /api/internal/match returns a valid Match Result object within the latency SLA for any valid request.
US-06	As an operations engineer, I want alerts when the pipeline degrades, so that I can respond before employers notice.	All alerts in Section 9 fire within 2 minutes of the triggering condition being met.

5. Functional Requirements

Requirements are classified as Must (non-negotiable for v1.0), Should (high-value, targeted for v1.0), or Could (deferred if time constrained). Each requirement is independently testable.

5.1 Embedding Generation

ID	Requirement	Priority	Test Method
FR-001	The pipeline must generate a vector embedding for every candidate profile using Google Gemini text-embedding-004 upon ingestion.	Must	Unit test: assert embedding shape and type on valid profile input.
FR-002	The pipeline must generate a vector embedding for every job post using Google Gemini text-embedding-004 upon ingestion.	Must	Unit test: assert embedding shape and type on valid job post input.
FR-003	The pipeline must store all embeddings in Chroma with the associated record ID as the metadata key.	Must	Integration test: retrieve embedding by record ID after ingestion.
FR-004	The pipeline must regenerate the embedding for any candidate profile or job post within 60 seconds of a record update.	Must	Integration test: update a record, assert new embedding differs from prior embedding within 60 seconds.
FR-005	The pipeline must cache embeddings and use the cached value when Gemini API is unavailable, provided the cache age is under 72 hours.	Must	Integration test: mock Gemini failure, assert cached embedding used; assert 503 returned when cache exceeds 72 hours.
FR-006	The pipeline should log a warning whenever a cached embedding is used in place of a freshly generated one.	Should	Log assertion: warning entry present after cache hit during Gemini outage.

5.2 Semantic Similarity Search

ID	Requirement	Priority	Test Method
FR-007	The pipeline must trigger a vector similarity search against all candidate embeddings whenever a new job embedding is stored.	Must	Integration test: store job embedding, assert search executes and returns a non-empty shortlist.
FR-008	The similarity search must return a shortlist of at least 20 candidates ranked by cosine similarity score, or all candidates if fewer than 20 exist.	Must	Unit test: assert shortlist length equals min(candidate_count, 20).
FR-009	The similarity search must carry the cosine similarity score forward as a feature into Phase 2 ranking.	Must	Unit test: assert feature vector for each shortlisted candidate contains the similarity score.

FR-010	The pipeline should allow the shortlist size (N) to be configured without code changes.	Should	Config test: change N via environment variable, assert shortlist length reflects updated value.

5.3 Feature Extraction

ID	Requirement	Priority	Test Method
FR-011	The feature extractor must compute a semantic skill similarity score for each candidate-job pair using all-MiniLM-L6-v2, comparing candidate skills against required skills.	Must	Unit test: assert skill score is higher for a candidate whose skills semantically match requirements than one with unrelated skills.
FR-012	The feature extractor must compute a separate semantic skill similarity score for nice-to-have skills.	Must	Unit test: assert nice-to-have score is non-zero when candidate has at least one skill semantically similar to a nice-to-have skill.
FR-013	The feature extractor must compute an experience gap value representing the difference between the candidate's years of experience and the job's minimum required years.	Must	Unit test: assert gap is zero or negative when candidate meets or exceeds minimum; positive when candidate falls short.
FR-014	The feature extractor must compute a salary overlap score reflecting the percentage overlap between the candidate's expected salary range and the employer's offered range.	Must	Unit test: assert 100 for full overlap, 0 for no overlap, proportional value for partial overlap.
FR-015	The feature extractor must compute a portfolio score of 0 when no portfolio is present, 30 when a portfolio URL exists but no projects are listed, and 100 when at least one project is listed.	Must	Unit test: three assertions covering each condition.
FR-016	The feature extractor must produce a complete numerical feature vector for every shortlisted candidate before passing it to the ranker.	Must	Unit test: assert feature vector contains all six features with no null values.

5.4 Candidate Ranking

ID	Requirement	Priority	Test Method
FR-017	The ranker must score each shortlisted candidate using the active XGBoost model and return an overall score between 0 and 100.	Must	Unit test: assert output score is a float between 0 and 100 inclusive for any valid feature vector.
FR-018	The ranker must return candidates in descending order of overall score.	Must	Unit test: assert output list is sorted descending by overall_score.

FR-019	The ranker must include a per-criterion score breakdown alongside the overall score in its output.	**Must**	Unit test: assert response object contains skills_score, experience_score, salary_score, portfolio_score for every result.
FR-020	On first deployment, the ranker must use a pre-trained XGBoost model initialised on the cold start dataset with default feature weights specified in Section 7.	**Must**	Integration test: deploy with cold start model, assert results differ materially from random ordering on validation dataset.
FR-021	The ranker must not use any model that has failed the NDCG@10 validation check defined in Section 7.4.	**Must**	Integration test: inject a deliberately poor model, assert system rejects it and retains the previous model.

5.5 Behavioral Signal Collection

ID	Requirement	Priority	Test Method
FR-022	The behavioral tracker must record which candidate profiles an employer opens, the rank position of each opened profile, and the timestamp of each open event.	**Must**	Integration test: simulate employer opening profiles, assert Layer 1 records match open events.
FR-023	The behavioral tracker must record the duration, in seconds, that an employer spends on each section of an opened profile, segmented by skills, experience, portfolio, and location.	**Must**	Integration test: simulate section dwell events, assert Layer 2 records contain correct durations per section.
FR-024	The behavioral tracker must record which profiles the employer marks as shortlisted.	**Must**	Integration test: simulate shortlist action, assert Layer 3 record is created.
FR-025	The behavioral tracker must not record any candidate PII in the raw behavioral store. All candidate references in behavioral records must use pseudonymous tokens.	**Must**	Security test: assert raw behavioral store contains no candidate names, emails, or other PII fields.
FR-026	The behavioral tracker must write all events to the relational database within 5 seconds of the event occurring.	**Must**	Integration test: generate event, assert DB record exists within 5 seconds.
FR-027	The behavioral tracker should support replay of missed events in the event of a write failure, using an in-memory queue with a maximum retry window of 10 minutes.	**Should**	Integration test: mock DB write failure, assert event is retried and written within retry window.

5.6 Feedback Processor and Iteration Logic

The feedback processor governs when and whether the XGBoost model is retrained. The flowchart below illustrates every decision branch.

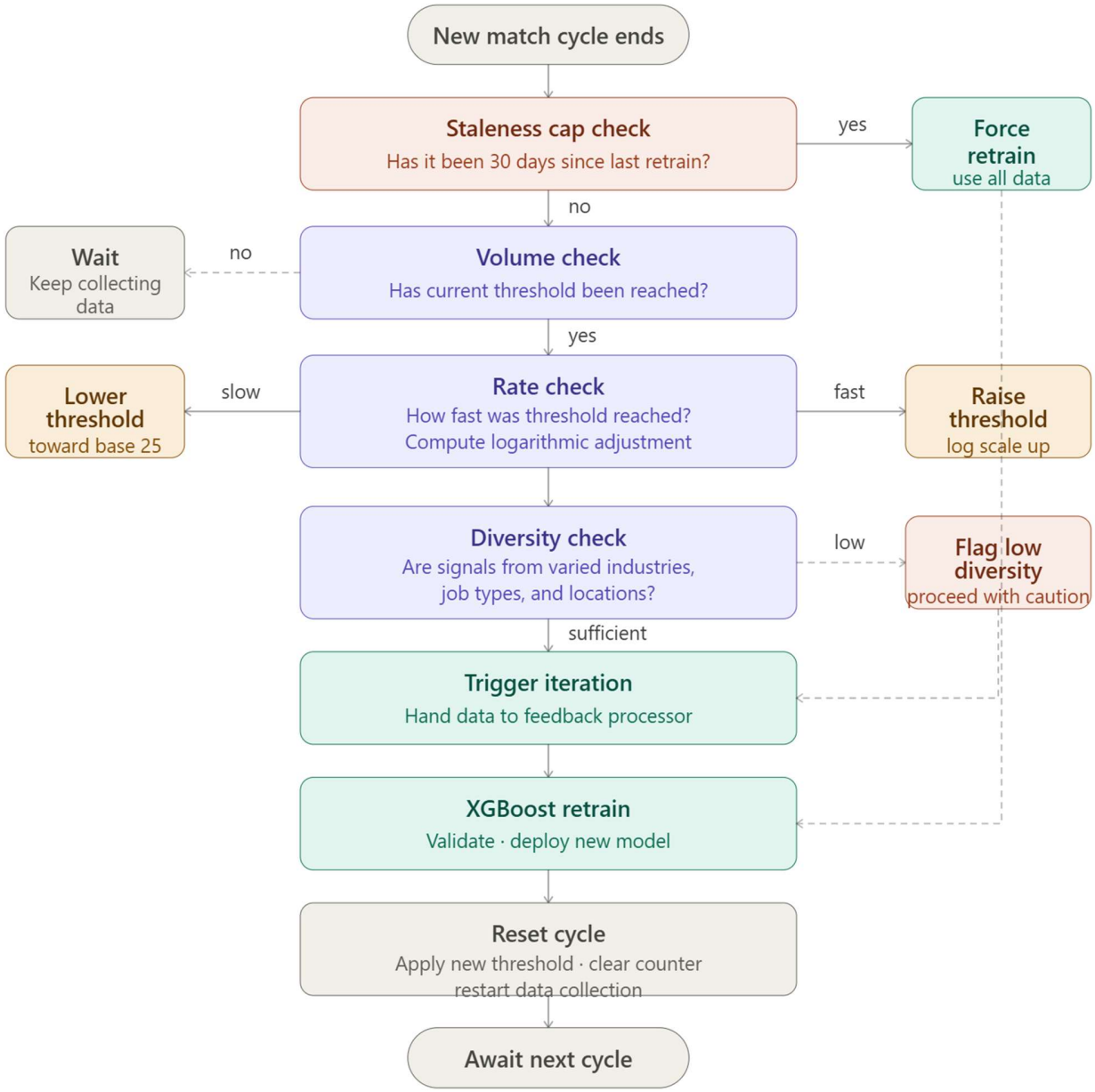

Figure 4. Iteration decision logic. The staleness cap fires first as a hard override. Volume, rate, and diversity checks follow. Both the forced retrain and the low-diversity paths converge into the main retraining flow.

ID	Requirement	Priority	Test Method
FR-028	The feedback processor must count completed employer sessions continuously. A session is complete when at least one Layer 3 shortlist event has been recorded for a job post.	Must	Unit test: assert session counter increments on Layer 3 event write.
FR-029	The feedback processor must trigger an iteration cycle when the accumulated session count reaches the current volume threshold.	Must	Unit test: set threshold to 5, simulate 5 sessions, assert iteration triggered.
FR-030	The volume threshold must start at 25 sessions and adjust logarithmically based on the rate at which the previous threshold was reached.	Must	Unit test: simulate fast accumulation, assert next threshold is higher than 25; simulate slow accumulation, assert threshold returns toward 25.
FR-031	The feedback processor must evaluate data diversity before triggering retraining. A batch is considered sufficiently diverse when it includes sessions from at least 3 distinct industries, 2 distinct job types, and 2 distinct locations.	Must	Unit test: low-diversity batch asserts flag is set; diverse batch asserts flag is absent.
FR-032	The feedback processor must trigger a forced iteration cycle if the active model has not been updated in 30 days, regardless of session count.	Must	Integration test: set last retrain timestamp to 31 days ago, assert forced iteration fires.
FR-033	The feedback processor must convert raw behavioral logs to structured training signals before passing data to the ML Training Module. It must not pass raw log rows directly.	Must	Unit test: assert training dataset schema differs from raw log schema.
FR-034	The feedback processor should log the diversity flag status, the session count, the rate of accumulation, and the trigger condition for every iteration event.	Should	Log assertion: iteration log entry contains all four fields.

5.7 API Endpoints

ID	Requirement	Priority	Test Method
FR-035	The pipeline must expose POST /api/internal/match, accepting { candidate_id, job_id } and returning a Match Result object as defined in Section 8.3.	Must	Integration test: valid request returns 200 with correct response schema.
FR-036	The pipeline must expose POST /api/internal/match/bulk, accepting { job_id } and returning an array of Match Result objects ordered by overall_score descending.	Must	Integration test: valid request returns 200 with ordered array.
FR-037	Both endpoints must return 400 for malformed requests, 404 when a referenced ID does not exist, and 503 when Gemini is unavailable and no valid cached embedding exists.	Must	Unit tests: one assertion per error condition per endpoint.

FR-038	Both endpoints must require a valid auth token in the Authorization header, provided by the backend auth service. Unauthenticated requests must return 401.	**Must**	Security test: request without token returns 401.
FR-039	The pipeline must expose API documentation via FastAPI's native OpenAPI interface at /docs.	**Must**	Smoke test: GET /docs returns 200.

6. Non-Functional Requirements

6.1 Performance

ID	Requirement	Priority	Test Method
NFR-001	Single match endpoint (POST /api/internal/match) must respond within 200ms at p95 under normal operating load.	Must	Load test: 100 concurrent requests, measure p95 latency.
NFR-002	Single match endpoint must not exceed 500ms at p99.	Must	Load test: measure p99 latency in same load test.
NFR-003	Bulk match against 10,000 candidates must complete within 30 seconds.	Must	Load test: 10,000 candidate corpus, measure total duration.
NFR-004	Bulk match must not exceed 60 seconds regardless of candidate corpus size within the supported scale.	Must	Load test: hard timeout assertion at 60 seconds.
NFR-005	Vector similarity search against 10,000 embeddings must complete within 100ms.	Must	Benchmark test: 10,000 embeddings in Chroma, measure search duration.
NFR-006	XGBoost inference for a single candidate must complete within 5ms.	Must	Benchmark test: 1,000 feature vectors, assert mean inference time under 5ms.
NFR-007	A full XGBoost retraining cycle must complete within 15 minutes.	Must	Retrain test: measure wall clock time from trigger to model deployment.

6.2 Reliability

ID	Requirement	Priority	Test Method
NFR-008	The pipeline must maintain 99.5% uptime for the match endpoints, excluding scheduled maintenance windows.	Must	Uptime monitor over 30-day period.
NFR-009	A Gemini API outage must not cause the match endpoints to return 5xx errors when a valid cached embedding exists for the requested records.	Must	Integration test: mock Gemini outage, assert 200 response using cached embeddings.
NFR-010	A failed retraining cycle must not affect the availability or behaviour of the production match endpoints.	Must	Integration test: inject retraining failure, assert match endpoints return unchanged results.
NFR-011	The previous XGBoost model must be retained for 14 days after a successful model replacement to allow rollback.	Must	Deployment test: assert prior model file persists for 14 days post-replacement.

6.3 Security

ID	Requirement	Priority	Test Method
NFR-012	All candidate IDs and employer IDs must be replaced with pseudonymous tokens before behavioral data enters the raw behavioral store.	Must	Security test: inspect raw store, assert no real IDs present.
NFR-013	The ID-to-token mapping must be stored separately from the behavioral store and must not be accessible to the ML Training Module.	Must	Architecture review: assert training module has no dependency on mapping table.
NFR-014	Raw behavioral logs must be purged after 12 months.	Must	Scheduled job test: assert logs older than 12 months are absent after purge job runs.
NFR-015	Candidate PII must not appear in any log output from the pipeline, including error logs and trace logs.	Must	Log scan: inject candidate with known PII, assert PII absent from all log outputs.
NFR-016	On a verified candidate deletion request, all Chroma embeddings for that candidate must be removed within 1 hour and behavioral log records for that candidate's token must be removed within 48 hours.	Must	Integration test: trigger deletion, assert embedding absent within 1 hour and logs absent within 48 hours.

6.4 Maintainability

ID	Requirement	Priority	Test Method
NFR-017	The XGBoost training random seed must be pinned and configurable via environment variable to ensure reproducible retraining runs.	Must	Reproducibility test: run training twice with same seed and data, assert identical model output.
NFR-018	The shortlist size N, volume threshold base, staleness cap duration, and NDCG@10 minimum threshold must each be configurable via environment variables without code changes.	Must	Config test: change each variable, assert pipeline behaviour changes accordingly.
NFR-019	All pipeline components must emit structured JSON logs with at minimum: timestamp, component name, record IDs (pseudonymised), phase, duration, and outcome.	Must	Log schema test: assert all fields present in sample log output.

7. ML Training Module

The ML Training Module runs on two occasions: at cold start initialisation and on every iteration cycle triggered by the feedback processor.

7.1 Cold Start Dataset

Before v1.0 goes to production, the team must produce a curated dataset of 200 candidate-job pairs with known correct match outcomes. Each pair must include a complete feature vector and a binary shortlist label. The dataset must span at least 5 distinct industries and 4 distinct job types. Delivery of this dataset is a launch blocker.

7.2 Default Feature Weights

The cold start model is initialised using the following feature weights. These weights serve as the starting point until behavioral data produces a trained replacement.

Feature	Default Weight	Rationale
Required skills match score	0.40	Primary hiring signal
Experience gap score	0.20	Hard constraint in most roles
Salary overlap score	0.20	Filters non-viable matches early
Portfolio score	0.10	Platform requirement per brief
Nice-to-have skills match score	0.10	Differentiator, not qualifier
Phase 1 similarity score	Carried as feature, not weighted directly	XGBoost learns its contribution

7.3 Training

XGBoost trains using a learning-to-rank objective. The training random seed is pinned per NFR-017. Each retraining run uses the full interpreted behavioral dataset from the feedback processor, not incremental updates.

7.4 Validation Criteria

A newly trained model must satisfy both conditions before replacing the production model:

21. NDCG@10 on the held-out behavioral test set must be at least 5% higher than the current production model's score on the same set.
22. NDCG@10 on the static cold start validation set must not be lower than the current production model's score on the same set.

A model that fails either condition is discarded. The current production model remains active. Three consecutive validation failures trigger a High-severity alert per Section 9.

7.5 Deployment

A model that passes both validation conditions replaces the active XGBoost instance. The prior model is retained for 14 days per NFR-011, then permanently deleted.

8. Data Contracts

The backend team must deliver data conforming to the schemas below. The pipeline validates on ingestion and rejects any record that does not conform. Rejection means a 400 response with a structured error body identifying the failing field.

8.1 Candidate Profile Schema

Field	Type	Required	Constraints
candidate_id	string	Yes	UUID format
skills	string[]	Yes	Minimum 1 item. Each item non-empty string.
years_of_experience	number	Yes	Integer, 0 or above.
salary_expectation.min	number	Yes	Positive integer.
salary_expectation.max	number	Yes	Positive integer, must be >= min.
salary_expectation.currency	string	Yes	ISO 4217 code (e.g. USD, KES, GBP).
portfolio.url	string	No	Valid URL or null.
portfolio.projects	object[]	No	Each project must contain title (string) and description (string).
extracted_text	string	Yes	Minimum 10 characters. Extracted from uploaded resume by backend team.
video_transcript	string	No	Plain text transcription from backend team. Null if no video was submitted.

8.2 Job Post Schema

Field	Type	Required	Constraints
job_id	string	Yes	UUID format.
employer_id	string	Yes	UUID format.
required_skills	string[]	Yes	Minimum 1 item.
nice_to_have_skills	string[]	No	Empty array permitted.
experience_level.min_years	number	Yes	Integer, 0 or above.
experience_level.max_years	number	No	Integer, must be >= min_years if provided.
salary_offered.min	number	Yes	Positive integer.
salary_offered.max	number	Yes	Positive integer, must be >= min.
salary_offered.currency	string	Yes	ISO 4217 code.
job_description_text	string	Yes	Minimum 50 characters.
portfolio_required	boolean	Yes	True or false.

8.3 Match Result Schema

Field	Type	Required	Constraints
candidate_id	string	Yes	UUID of matched candidate.
job_id	string	Yes	UUID of matched job post.
overall_score	number	Yes	Float, 0 to 100 inclusive.
breakdown.skills_score	number	Yes	Float, 0 to 100.
breakdown.experience_score	number	Yes	Float, 0 to 100.
breakdown.salary_score	number	Yes	Float, 0 to 100.
breakdown.portfolio_score	number	Yes	One of: 0, 30, or 100.
matched_at	string	Yes	ISO 8601 UTC timestamp.

9. Monitoring and Alerting

All alerts route to the on-call engineer. Every pipeline request must emit structured JSON logs containing: timestamp, component, record IDs (pseudonymised), phase, duration, and outcome.

ID	Trigger Condition	Action Required	Severity
MON-01	p95 match latency exceeds 500ms over any 5-minute window.	**Investigate pipeline bottleneck. Check Chroma and Gemini API response times.**	High
MON-02	Bulk match job exceeds 60 seconds.	Halt job. Check candidate corpus size and XGBoost inference time.	**High**
MON-03	Gemini API error rate exceeds 2% over any 1-hour window.	**Check Gemini API status. Verify cache is serving as fallback.**	High
MON-04	Any cached embedding older than 72 hours is used in a production match.	Investigate Gemini API availability and cache refresh job.	**Medium**
MON-05	Mean overall_score shifts more than 15 points against the 7-day rolling average.	**Review last retraining cycle. Check for data distribution shift.**	Medium
MON-06	ML Training Module exits with an error or exceeds 15 minutes.	Review training logs. Roll back to previous model if degraded.	**High**
MON-07	New model fails NDCG@10 validation 3 consecutive times.	**Manual investigation of training data quality and feature distribution.**	High
MON-08	More than 50% of iteration batches flagged as low-diversity over any 90-day window.	Review platform geographic and industry distribution. Consider threshold adjustment.	**Medium**
MON-09	Staleness cap of 30 days is reached and a forced iteration fires.	**No immediate action required. Log and monitor.**	Low

10. System Diagrams

10.1 System Context

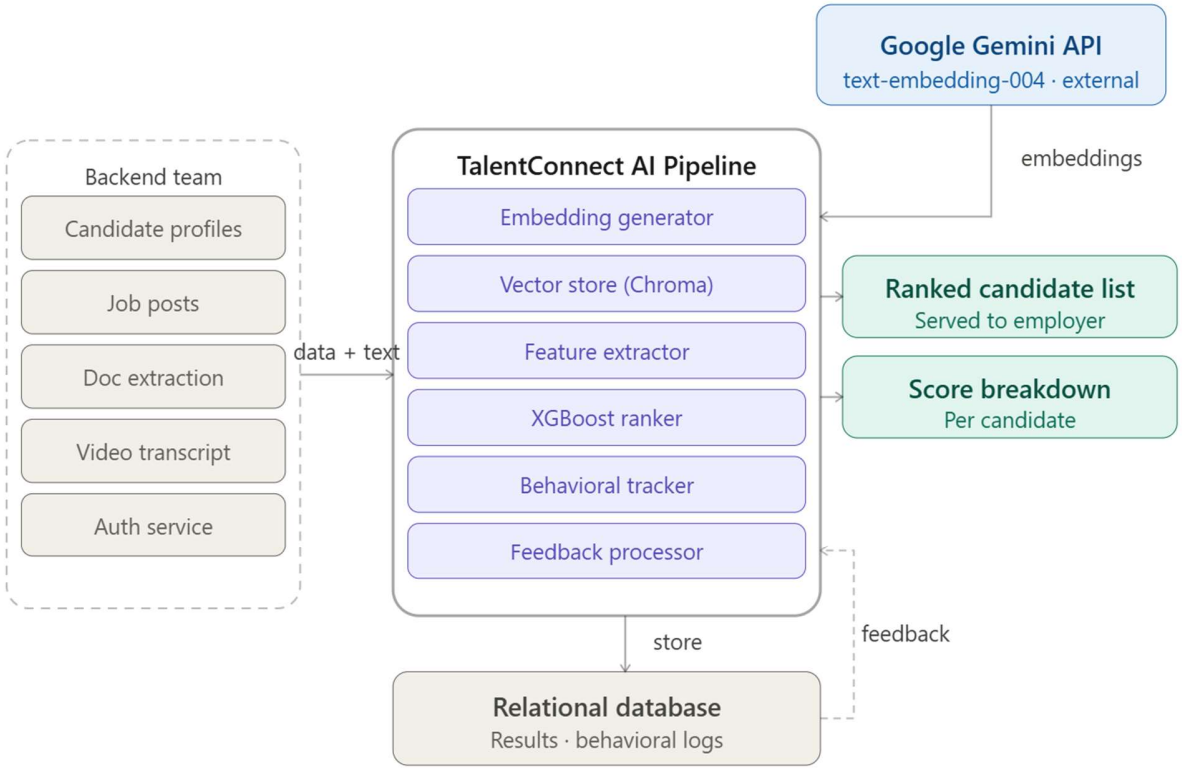

Figure 1. The TalentConnect AI Pipeline within the broader system. The dashed region is backend team territory. The solid-bordered region is our scope. Gemini is an external service.

10.2 Matching Pipeline

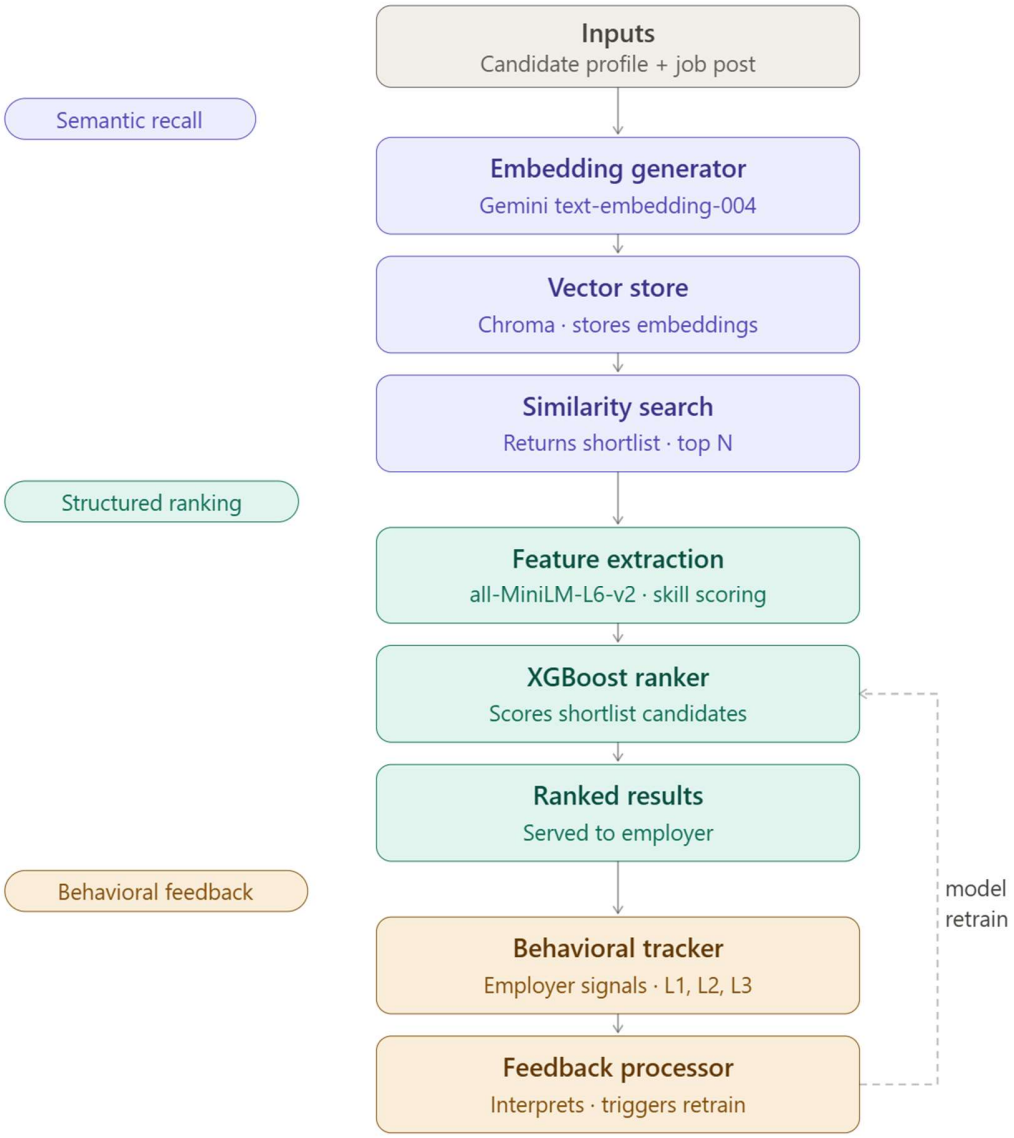

Figure 2. The three-phase pipeline: semantic recall via Gemini embeddings and Chroma vector search, structured ranking via feature extraction and XGBoost inference, and behavioral feedback driving iterative retraining.

10.3 Behavioral Feedback Loop

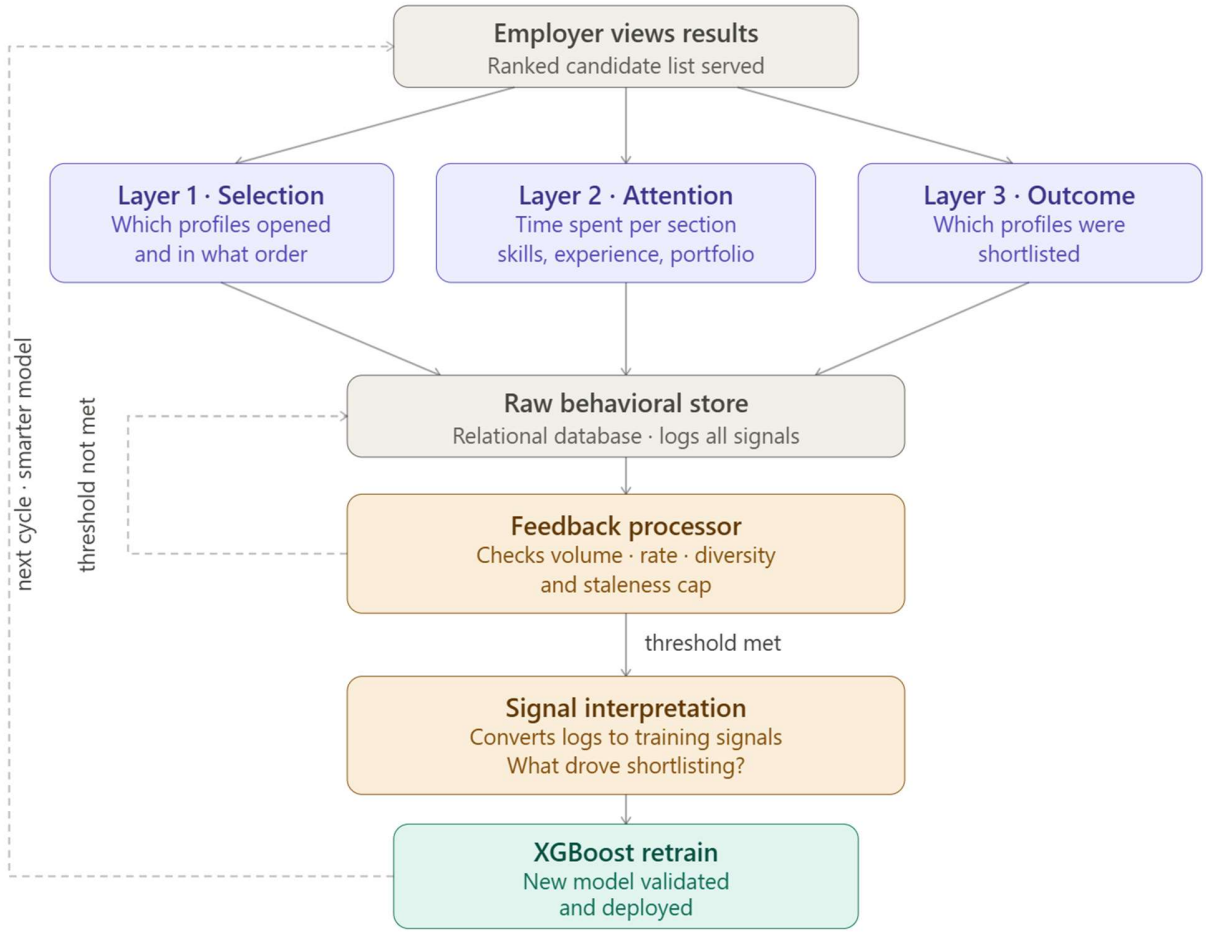

Figure 3. Employer interaction generates Layer 1, Layer 2, and Layer 3 signals. All three converge into the raw behavioral store. The feedback processor interprets the data and, once iteration conditions are met, triggers XGBoost retraining. The retrained model feeds back into the next matching cycle.

10.4 Adaptive Threshold Logic

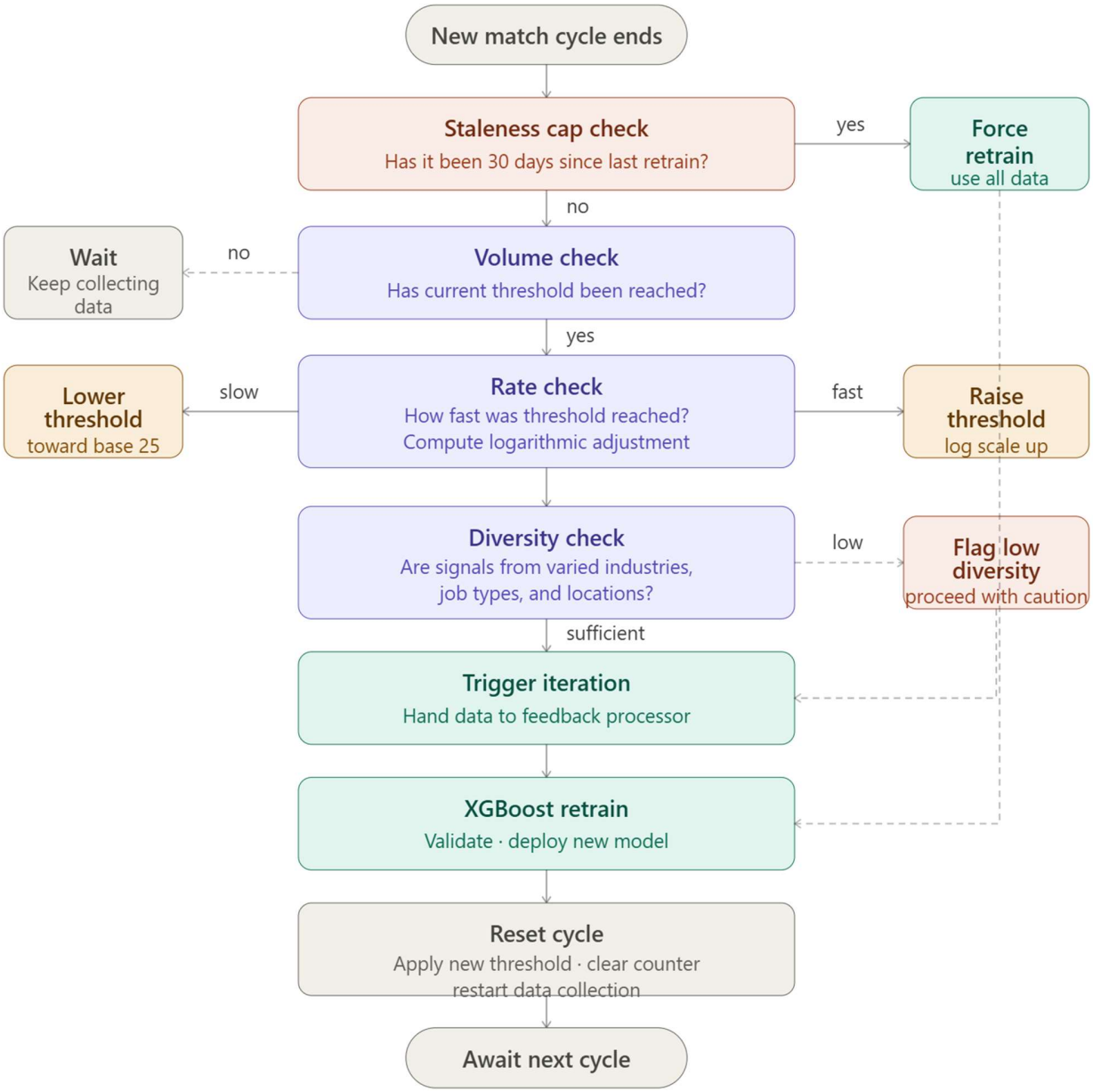

Figure 4. The staleness cap fires first as a hard override. Volume, rate, and diversity checks follow in sequence. Both the forced retrain path and the low-diversity path converge into the main retraining flow before cycle reset.

11. Assumptions and Constraints

11.1 Assumptions

23. The backend team will deliver sanitized candidate profile and job post data fully conforming to the schemas in Section 8 before the matching pipeline is called.
24. Document text extraction and video transcription are complete before data reaches the pipeline. The pipeline does not support binary data.
25. The relational database provisioned by the backend team is accessible from the matching pipeline's deployment environment.
26. Google Gemini text-embedding-004 is available at the API endpoint specified in its current documentation. Rate limits are sufficient for launch volume.
27. The cold start dataset of 200 labelled pairs will be ready before the launch date.

11.2 Constraints

28. The entire technology stack must use free or open-source tooling. No paid ML infrastructure is permitted for v1.0.
29. The matching algorithm must not expose job post content to candidates or candidate profile content to employers at any point.
30. All behavioral data must be pseudonymised before any processing. No real IDs may enter the training pipeline.
31. Chroma runs locally alongside the chosen relational database. No managed vector database service is used for v1.0.

12. Dependencies

12.1 Internal Dependencies

Dependency	Owner	Blocks	Status
Candidate profile service delivering conforming JSON	Backend team	FR-001, FR-003	Open
Job post service delivering conforming JSON	Backend team	FR-002, FR-004	Open
Document text extraction from uploaded resumes	Backend team	FR-001	Open
Video transcription from candidate introductions	Backend team	FR-001	Open
Auth service for validating internal API tokens	Backend team	FR-038	Open
Relational database provisioned and reachable	Backend team	FR-026, FR-035	Open
Chroma provisioned in deployment environment	Infra / Backend team	FR-003, FR-007	Open
Cold start dataset of 200 labelled pairs	Matching algorithm team	FR-020	Open

12.2 External Dependencies

Google Gemini text-embedding-004 is the only external service dependency. Two facts about it require team confirmation before launch. First, the free tier rate limits must accommodate expected daily embedding generation volume at launch, accounting for both new record ingestion and cache refresh cycles. Second, a sustained Gemini outage lasting more than 72 hours will degrade pipeline quality because cached embeddings expire. The team must decide and document an acceptable degraded-mode policy for that scenario before go-live.

12.3 Technology Stack

Component	Technology	Licence
API framework	Python / FastAPI	MIT / Open source
Embedding model	Google Gemini text-embedding-004	Free within rate limits
Semantic skill comparison	all-MiniLM-L6-v2 via HuggingFace	Apache 2.0 / Open source
Vector database	Chroma	Apache 2.0 / Open source
Ranking model	XGBoost	Apache 2.0 / Open source
Database interface	SQLAlchemy	MIT / Open source
API documentation	FastAPI native OpenAPI / Swagger	Included with FastAPI

13. Open Items

#	Item	Owner	Blocks	Status
1	Confirm Gemini API free-tier rate limits are sufficient for projected launch volume and 90-day growth.	Backend team	FR-001, FR-002	Open
2	Confirm relational database selection (MySQL or PostgreSQL) so SQLAlchemy dialect can be configured.	Backend team	FR-026, FR-035	Open
3	Define and agree on the JSON event schema for behavioral tracker payloads before backend team begins instrumentation.	Both teams	FR-022, FR-023, FR-024	Open
4	Produce the cold start dataset of 200 labelled candidate-job pairs, spanning at least 5 industries and 4 job types.	Matching algorithm team	FR-020	Open
5	Confirm Chroma will be provisioned in the deployment environment alongside the relational database.	Infra / Backend team	FR-003, FR-007	Open
6	Define and document the degraded-mode policy for a Gemini outage exceeding 72 hours.	Both teams	NFR-009	Open
7	Confirm data retention and deletion policy with compliance or legal stakeholders before behavioral tracking goes live.	TalentConnect leadership	NFR-014, NFR-016	Open

End of Document | TalentConnect AI Matching Pipeline | PRD v1.0 | 17 March 2026